3.

How do Genetic Algorithms Work?

The slides follow the lines of two monographs: Z. Michalewicz *Genetic Algorithms+Data Structures=Evolution Programmes* and A.Eiben&J.Smith *Introduction to Evolutionary Computing*

Implementation and Use of a GA

1. Define its elements
   - Representation
   - Fitness function
   - Selection mechanism
   - Genetic operators
   - Parameters
2. Design the experiment (presuposes the optimisation of the GA)
   - Stand-alone performance of the GA
   - Comparison of the GA against other (non-GA) algorithms
3. Actually perform the experiment
4. Interpret the results

Estimating the stand-alone performance of a GA

- The quality of a probabilistic algorithm can only be measured as **statistics** resulted from **experiments**.
- Example: the **average** of the best solution in each of **n** runs of the algorithm
  - Or the **best solution** in **n** runs.
- Subsequently, the statistical **parameter** *solution_found_by_the_GA* (be it, as above, the average, the best - or any other performance indicator) can be estimated through **confidence intervals** or by one-sample **hypothesis testing**.
- The consistency of the GA's behaviour can be estimated similarly, using the standard deviation of the statistics.

Comparing Two Probabilistic Algorithms

- The relative quality of a GA against another probabilistic algorithm (Hillclimbing, Simulated Annealing, etc.) can be studied empirically by **hypothesis testing**.
  - At least 30 runs of each algorithm allow for applying the **Student t-test**, for example.
- If a significant difference in averages is found, then one algorithm can be stated to perform better than the other.
- For the comparison of three or more probabilistic algorithms, **analysis of variance** should be used.
  - The algorithms to be compared have to be individually **optimised before** the experiment

Is the Problem Fit to Be Solved with a GA ?

1. Can the problem be stated as optimisation of a real function?
2. Is a step-by-step solution improvement strategy possible?
   - Otherwise, create a frame for this (e.g., satisfiability CNF: number of minterms evaluated to **1**)
3. Is it a uni-criterial optimisation? Otherwise:
   - build one global criterion (linear – timetable –/ non-linear) or
   - work in a Pareto (Nash…) setting
4. Is it a maximization problem? Otherwise:
   - use **-f** or **1/f** or analogues, instead of **f**
5. Is it optimisation of a positive function?
   - Otherwise, translate the function by an appropriate constant
6. Is it optimisation with restrictions?
   - The three approaches: encoded in representation; penalties; repair.

Example: the Satisfiability Problem

- Straightforward to encode solutions (a string of k bits is a truth assignment for k variables)
- Straightforward evaluation (the truth value of the resulting expression)
- But with only two possible values of the fitness function, impossible to improve step by step.
  - A non-optimal truth-assignment gives no hint as to how to change it into a better candidate solution.
- Fitness for conjunctive normal form: number of terms which are evaluated as "true"
  - Thus, a step-by-step improvement becomes possible.

A GA Frame for Numerical Optimisation

A GA Frame for Numerical Optimisation

- **General problem:** find the optimal (maximal) value of the function **f**: $\mathbf{R}^k \rightarrow \mathbf{R}$, with restrictions $x_i \in D_i = [a_i ; b_i]$, $f > \mathbf{0}$.
- Requirement: solution with 6 decimal positions.

- **The elements of a numerical GA**

1. Representation.

- D_i is to be cut into 2^{m_i} subintervals, where $(b_i - a_i) \cdot 10^6 \le 2^{m_i} - 1$, with m_i minimal
- Each x_i would then be represented by a substring of m_i bits: $x_i = a_i + decimal(d_{m_i-1} \ldots d_1 d_0) \cdot \frac{b_i - a_i}{2^{m_i} - 1}$
- A candidate solution is <u>represented</u> by a bitstring of length

$$m = \sum_{i=1}^{k} m_i .$$

Decoding a Bitstring

- $decimal\ (d_{mi-1} \ldots d_1 d_0) = \Sigma_{j=0}^{mi-1} d_i * 2^i < 2^{m_i} - 1$
- $0 \le decimal\ (d_{mi-1} \ldots d_1 d_0) \le 2^{m_i} - 1$
- $0 \le decimal\ (d_{mi-1} \ldots d_1 d_0) / (2^{m_i} - 1) \le 1$
- $0 \le decimal\ (d_{mi-1} \ldots d_1 d_0) * (b_i - a_i) / (2^{m_i} - 1) \le (b_i - a_i)$
- $a_i \le x_i = a_i + decimal(d_{m_i-1} \ldots d_1 d_0) \cdot \frac{b_i - a_i}{2^{m_i} - 1} \le b_i$

The Other Elements of a GA for Numerical Optimisation

2. Initialization:

- population $v_1, v_2, \ldots, v_{pop_size}$.
- random bitwise or
- heuristic (some better-than-worst solutions, using properties of **f**).

3. Evaluation:

- decode the bitstring **v** into a vector **x** of numbers
- calculate **eval(v)=f(x)** .

4. Halting condition: a maximum number of generations (alternatively: generated chromosomes) may be enforced.

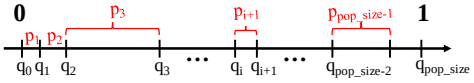
5. Selection.

- Probability field based on fitness values
- Best known procedure is "*fortune wheel*":
  - Slots proportional to fitness.
  - Calculate (steps 1., 2., 3. ,4. below, for i=1..*pop_size*)
    1. Individual fitnesses: for each chromosome $v_i \rightarrow$ **eval**(v_i)=**f**(x_i)
    2. Total fitness: $F = \sum_{i=1}^{pop_{size}} eval(v_i)$
    3. Individual selection probabilities: p_i=**eval**(v_i) / **F**

    At this step, the probability field is already built.

    Other options for selection: deterministic, rank-based, tournament etc.
    4. Cumulative selection probabilities: $q_i = \sum_{j=1}^{i} p_j$, $q_0 = 0$.
- Selection procedure: the *fortune wheel* is "rotated" *pop_size* times.

Probability field and cumulative probabilities



5. **Selection** for Survival (continued)

- Idea: create an **intermediate population**
  - Spin the fortune wheel *pop_size* times. Each time:
    - select **for survival** one chromosome from the current population
    - put it into the intermediate population.
- **Implementation:**
  - ***generate*** **a random (float) number *r* in [0 , 1];**
  - **if ($q_{i-1} < r \leq q_i$) then *select* v_i.**
- With higher probability:
  - best chromosomes get more and more copies;
  - average stay even;
  - worst die off.

6. **Operators** for a Numerical Optimisation GA:

a) Crossover

- p_c – the probability of crossover – gives the expected number of chromosomes which undergo crossover: $p_c \cdot$*pop_size*.
- **Selection for crossover** (→parents).
  - In the intermediate population

for (each chromosome) ***do***
  generate a random (float) number *r* in [0 , 1];
  if $r < p_c$ ***then*** *select* the current chromosome.
apply crossover to the selected **parents**, randomly paired.

Crossover (continued)

- ***for*** (each paired parents) ***do***
  - *generate* a random number $pos \in \{1,..,m-1\}$
  - *replace* the two parents by their two offspring.

$$P1 = (a_1\ a_2 \mid \ldots a_{pos}\ a_{pos+1} \ldots a_m)$$
$$P2 = (b_1\ b_2 \mid \ldots b_{pos}\ b_{pos+1} \ldots b_m)$$

$$O1 = (a_1\ a_2 \ldots a_{pos}\ b_{pos+1} \ldots b_m)$$
$$O2 = (b_1\ b_2 \ldots b_{pos}\ a_{pos+1} \ldots a_m)$$

- Uniform distribution for generating the random numbers? See *Evolving crossover hotspots* (Schaffer&Morishima)

6. **Operators** for a numerical Optimisation GA

b) Mutation

- $p_{mutation}$ – probability (rate) of mutation.
- The *expected* number of mutated bits in one generation: $p_{mutation} \cdot m \cdot$*pop_size*
- Each bit in the population has the same chance to undergo mutation.
  - Sometimes, targeted distributions (other than uniform) are useful

for (each chromosome <u>after crossover</u>) ***do***
  for (each bit within the chromosome) ***do***
    generate a random number *r* in [0 , 1];
    if (*r* < $p_{mutation}$**)** ***then*** *mutate* the current bit.

A Specific Numerical Example (Michalewicz)

max **f**

$\mathbf{f}(x_1,x_2) = 21.5 + x_1 \cdot \sin(4\pi x_1) + x_2 \cdot \sin(20\pi x_2)$,
with restrictions: $-3.0 \leq x_1 \leq 12.1$, $4.1 \leq x_2 \leq 5.8$,
with 4 decimal places for each variable.

- **Representation**:
  - $2^{17} < 151000 < 2^{18}$; $2^{14} < 17000 < 2^{15}$: 33 bits.
  - **v** = (01000100101101000 0.111110010100010)
  - 010001001011010000→1.0524
  - 111110010100010→5.7553
  - **eval**(**v**) = **f**(1.0524, 5.7553) = 20.2526.
- **Parameters**: *pop_size* = 20; $p_c = 0.25$; $p_m = 0.01$.

First Generation

Random initialisation

Evaluation of generation 0:
best_of_generation was v_{15}; worst was v_2

- $F = \sum \mathbf{eval}(v_i) = 387.7768$
- Field of probabilities (p_i, i=1..*pop_size*).
- Cumulative probabilities (q_i, i=0..*pop_size*).
- Spinning the fortune wheel *pop_size* times.
- New population.

First Generation - Crossover

- **Selection for crossover**.
  - Some chromosomes selected (on the average, $p_c \cdot pop_size$).
  - ***If*** (odd number) ***then*** randomly *decide* {drop / add}.
  - Randomly couple together selected parents.
    - No need for new random numbers – use the ranking of those $r < p_c$, already generated, to sequence and mate selected chromosomes
- **Example – one pair**. Generate $pos \in \{1..32\}$. *Pos* = 9.

- **v_2'= (100011000.10110100111100000111001 0)**
- **v_{11}'= (111011101.10110000100011111011110)**

- **v_2"= (100011000. 10110000100011111011110)**
- **v_{11}"=(111011101. 10110100111100000111001 0)**

First Generation – Mutation

- **Selection for mutation**.
  - 660 times generate a random number in [0;1]
  - Distribution (uniform or targeted)
  - Expected number of mutated bits: $p_m \cdot m \cdot pop_size$=6.6

r	Bit position in the whole population	Chromosome number	Bit position in the chromosome
0.000213	112	4	13
0.009945	349	11	19
0.008809	418	13	22
0.005425	429	13	33
0.002836	602	19	8

Outcomes and Conclusions

- Best chromosome in the final, 1000th generation: v_{11}=(11010110000001001000110001011 0000)
  eval (v_{11}) = f (9.6236 , 4.4278) = 35.4779.
- However, in the 396th generation:

eval (*best_of_generation*) = 38.8275 !

- Indeed, sometimes, the best chromosome is not selected.
- Solution for not loosing the best chromosome:

Store *best_so_far* (not *best_of_generation*) as the current solution given by the GA.

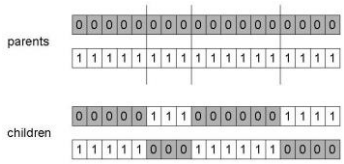
  - *best_so_far* is initialised with the best chromosome in the first generation
  - *best_so_far* is updated after each generation, if a better chromosome is found
    - operators, selection, evaluation are not applied to *best_so_far*

Alternative crossover and mutation operators

The remaining slides follow: Eiben &Smith – *Introduction to Evolutionary Computing*

k-point crossover

- Choose **k** random crossover points
- Split at those points
- Glue parts, alternating between parents
  - Generalisation of 1 point crossover
- Uniform crossover, for **k**=*chromosome_length*-1



Crossover operators for **real valued** genes

- Discrete:
  - each allele value in offspring *z* comes from one of its parents *(x,y)* with equal probability: $z_i = x_i$ or y_i
    - Random, uniform
- Intermediate
  - exploits the idea of creating children "between" parents: *arithmetic* recombination
  - $z_i = \alpha x_i + (1 - \alpha) y_i$ where $\alpha : 0 \le \alpha \le 1$.
  - The parameter α can be:
    - constant: uniform arithmetical crossover
    - variable (e.g., depending on the age of the population)
  - Crossover on **fenotype**

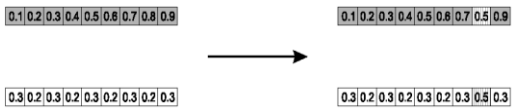
Real-valued genes: **Single arithmetic** crossover

- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- Pick a single gene (*k*) at random,
- First offspring:

$$\langle x_1, \ldots, x_k, \alpha \cdot y_k + (1-\alpha) \cdot x_k, \ldots, x_n \rangle$$

reverse for the other offspring: (1-⟨ , ⟨)

For technical simplicity, in the illustration below, $\alpha = 0.5$



Real-valued genes: **Simple arithmetic** crossover

- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- Pick random gene *(k)* and after this *locus*, mix values
- First offspring:

$$\langle x_1, \ldots, x_k, \alpha \cdot y_{k+1} + (1-\alpha) \cdot x_{k+1}, \ldots, \alpha \cdot y_n + (1-\alpha) \cdot x_n \rangle$$

reverse for the other offspring: (1-⟨ , ⟨)

For technical simplicity, in the illustration below, $\alpha = 0.5$

Real-valued genes: **Whole arithmetic** crossover

- Most commonly used
- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- First offspring:

$$\alpha \cdot \bar{x} + (1-\alpha) \cdot \bar{y}$$

reverse for the other offspring: (1- α, α)

For technical simplicity, in the illustration below, $\alpha = 0.5$

Permutations: **Insert Mutation**

- Pick two genes at random
- Move the second to follow the first, shifting the rest along to accommodate
- Note that this preserves most of the order and the adjacency information

Permutations: **Inversion mutation**

- Pick two genes at random and then invert the substring between them.
- Preserves most adjacency information (only breaks two links), but disruptive of order information

Permutations: **Scramble mutation**

- Pick a subset of genes at random
- Randomly rearrange the alleles in those positions

- The subset of genes (actually, **loci**) does not have to be contiguous

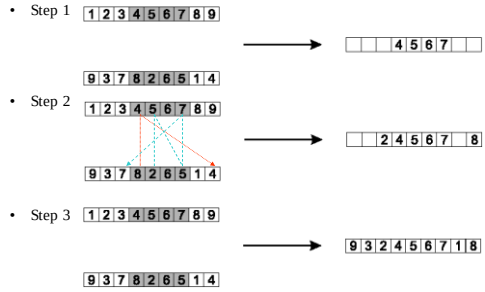
Permutations: **Partially Mapped** Crossover (PMX)

Informal procedure for the first offspring of parents P1 and P2:

1. Choose random segment and copy it from P1
2. Starting from the first crossover point, look for **elements in that segment of P2 that have not been copied**
3. For each of these *i*, look in the offspring to see what element *j* has been copied in its place from P1
4. Place *i* into the position occupied *j* in P2, since we know that we will not be putting *j* there (as it is already in offspring)
5. If the place occupied by *j* in P2 has already been filled in the offspring *k*, put *i* in the position occupied by *k* in P2
6. Having dealt with the elements from the crossover segment, the rest of the offspring can be filled from P2, on their respective positions.

The second offspring is created analogously.

Permutations: PMX example



Permutations: **Cycle crossover**

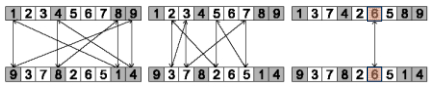
Basic idea: Each gene comes from one parent *together with its position*.

Informal procedure:

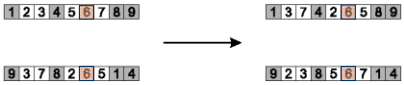
1. While there are unvisited genes in P1, make cycles of genes from P1 in the following way.
   (a) Start with the first unvisited gene of P1.
   (b) Look at the gene at the *same position* in P2.
   (c) Go to the position with the *same allele* in P1.
   (d) Add this allele to the cycle.
   (e) Repeat step (b) through (d) until you arrive at the first gene of P1.
2. While there are unused cycles, alternate pieces of cycles from the **first** and then **second** parent in the **first** offspring on the positions they have in the respective parents.
3. While there are unused cycles, alternate pieces of cycles from the **second** and then **first** parent in the **second** offspring on the positions they have in the respective parents.

Cycle crossover example

- Step 1: identify all cycles between parents



- Step 2: copy cycles into the two offspring, alternating the parents while successive cycles are copied.



Graphs: **Edge Recombination**

- Candidate solutions: circuits in a graph.
- **Two parents, one offspring**.
- Works by constructing a table listing which edges are present in the two parents. If an edge is common to both, mark with a +
- Example: two candidate circuits

(1 2 3 4 5 6 7 8 9); (9 3 7 8 2 6 5 1 4)

Element	Edges	Element	Edges
1	2,5,4,9	6	2,5+,7
2	1,3,6,8	7	3,6,8+
3	2,4,7,9	8	2,7+, 9
4	1,3,5,9	9	1,3,4,8
5	1,4,6+		

Graphs: Edge Recombination (continued)

Informal procedure once the edge table above is constructed.

1. Pick an initial element at random and put it in the offspring
2. Set the variable current element = entry
3. Remove from the table all references to the current element
4. Examine in the table the row of the current element:
   – If there is a common edge (marked with "+"), pick that to be next element
   – Otherwise pick the entry in the list whose row has the shortest list
   – Ties are split at random
5. In the case of reaching an empty list, a new element is chosen at random

Graphs: Edge Recombination example

Element	Edges	Element	Edges
1	2,5,4,9	6	2,5+,7
2	1,3,6,8	7	3,6,8+
3	2,4,7,9	8	2,7+, 9
4	1,3,5,9	9	1,3,4,8
5	1,4,6+		

Choices	Element selected	Reason	Partial result
All	1	Random	[*1*]
2,5,4,9	5	Shortest list	[*1 5*]
4,6	6	Common edge	[*1 5 6*]
2,7	2	Random choice (both have two items in list)	[*1 5 6 2*]
3,8	8	Shortest list	[*1 5 6 2 8*]
7,9	7	Common edge	[*1 5 6 2 8 7*]
3	3	Only item in list	[*1 5 6 2 8 7 3*]
4,9	9	Random choice	[*1 5 6 2 8 7 3 9*]
4	4	Last element	[*1 5 6 2 8 7 3 9 4*]